

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	23
REFERÊNCIAS.....	24

EMSE

O QUE VEM POR AÍ?

Você sabe o que é e qual é a diferença de Dockerfile e Docker Compose? E como essas tecnologias são utilizadas no mercado de trabalho?

Nesta aula, você aprenderá sobre ambos. Além disso, irá conhecer as boas práticas na otimização de imagens Docker, explorar funcionalidades avançadas do Dockerfile, realizar deploy de um projeto de machine learning usando Dockerfile e Streamlit. Vamos ainda explorar funcionalidades avançadas do Docker Compose e realizar deploy de outro projeto de machine learning no Kubernetes do Google Cloud Platform (GKE) usando Docker Compose e Kompose.

HANDS ON

Nessa aula você vai aprender como o Dockerfile e o Docker Compose funcionam, utilizar volumes, otimização de imagens, criar sua conta no GCP, realizar deploys, conhecer o GKE e mais.



SAIBA MAIS

Reforçando conceitos do Dockerfile e Docker Compose

Antes de explicar o que de fato é Dockerfile, irei utilizar uma analogia para melhor entendimento. Imagine o Dockerfile como uma receita, em que temos os ingredientes, procedimentos e passos a serem seguidos. Para que a receita seja executada com sucesso temos que seguir um passo a passo, em que colocamos um ingrediente X, fazemos um procedimento Y e uma sequência de passos N até chegar ao final.

Sendo assim, o Dockerfile, assim como uma receita, é também um arquivo declarativo no qual realizamos uma sequência de passos e procedimentos para construir uma imagem Docker.

Visando o mundo real no mercado de trabalho, imagine ter que digitar dezenas de comandos Docker para construir imagens e containers manualmente. Cansativo, não? E ainda é preciso compartilhar o mesmo processo com demais colegas de equipe, o que pode gerar erros por serem processos manuais e ter um alto tempo de configuração.

Visando esse problema, foi construída a solução que hoje se chama Dockerfile: por meio desse arquivo, podemos adicionar comandos específicos para realizar N procedimentos. Sendo assim, trata-se de um automatizador de criação de imagens que facilita o compartilhamento de imagens e diminui o tempo de configuração e manutenção.

Usando a analogia novamente, se o Dockerfile é uma receita em que temos N passos a serem seguidos para construir uma imagem, podemos dizer que o Docker Compose é uma opção no menu com alguns itens para aquela opção. Sendo assim, para cada item temos procedimentos diferentes, pois cada um deles é uma imagem ou container. O Docker Compose, assim como um menu, é um arquivo declarativo em que adicionamos as opções (ou serviços) com os itens (ou volumes, imagens, portas) que queremos que essa opção tenha.

Agora, visando o mundo real no mercado de trabalho, imagine uma integração de N serviços, aplicações, bancos e demais ferramentas. Isso pode acabar se

tornando trabalhoso, além de desafiador, se utilizarmos somente Dockerfile ou comandos manuais visando escalabilidade de projetos.

Em cenários parecidos com esse, o Docker Compose surgiu para facilitar a escalabilidade e a replicação de ambientes de uma forma mais segura e padronizada, pois não é necessário realizar todos aqueles comandos manuais no terminal. Sendo assim, ganhamos tempo de desenvolvimento e diminuimos o tempo de configuração de projetos.

Dockerfile

Esse arquivo é onde declaramos a sequência de passos que devemos executar no processo de criação da imagem. Antes de falarmos como construímos imagens com Dockerfile, é importante abordar como criamos um arquivo Dockerfile. Sendo assim, deixo segue um exemplo:

```
FROM python:3.8-slim

USER root

ARG APP_VERSION=1.0

LABEL maintainer="root@fiap.com"

ENV APP_ENV dev

WORKDIR /app

RUN apt-get update && \
    apt-get install -y gcc

COPY . /app

RUN pip install -r requirements.txt

HEALTHCHECK CMD curl --fail http://localhost:8080/ || exit 1

VOLUME ["/data"]

EXPOSE 8080

ENTRYPOINT ["streamlit", "run", "app.py"]

CMD []
```

Código-fonte 1 – Exemplo de um arquivo Dockerfile com alguns dos comandos mais utilizados
Fonte: Elaborado pelo autor (2024)

Esse é o arquivo declarativo de comandos para criação de imagens e é importante reforçar que toda imagem customizada provém de uma imagem base existente. Agora, visando o arquivo declarativo, repare que para cada linha temos um comando específico que detém alguma função. Para deixar mais claro o que cada comando faz, segue uma lista.

- **FROM:** serve para definir a imagem base da nossa imagem customizada, em que podemos também selecionar qual versão queremos.

```
FROM python:3.8-slim
```

Código-fonte 2 – Exemplo de comando que seleciona imagem base no Dockerfile
Fonte: Elaborado pelo autor (2024)

- **USER:** aqui podemos definir o tipo de usuário que realizará comandos dentro do container; se for root, tem mais privilégios do que o tipo user, por exemplo.

```
USER root
```

Código-fonte 3 – Exemplo de comando que seleciona o tipo de usuário base no Dockerfile
Fonte: Elaborado pelo autor (2024)

- **ARG:** é uma variável de ambiente em que seu tempo de vida é durante o momento de construção da imagem. Após a criação, não é possível acessar essa variável de ambiente.
 - Um exemplo de caso de uso seria utilizar uma senha, um token, algo temporário somente para a construção da imagem.

```
ARG APP_VERSION=1.0
```

Código-fonte 4 – Exemplo de comando que adiciona variáveis de ambiente temporárias durante criação de imagens no Dockerfile
Fonte: Elaborado pelo autor (2024)

- **LABEL:** serve para adicionar metadados à imagem.

```
LABEL maintainer="root@fiap.com"
```

Código-fonte 5 – Exemplo de comando que adiciona metadados na imagem via Dockerfile

Fonte: Elaborado pelo autor (2024)

- **ENV:** é uma variável de ambiente que continua existindo após o processo de construção da imagem na qual podemos acessar no modo de execução iterativo do container.

```
ENV APP_ENV dev
```

Código-fonte 6 – Exemplo de comando que adiciona variáveis de ambiente na imagem via Dockerfile

Fonte: Elaborado pelo autor (2024)

- **WORKDIR:** serve para definir o diretório de trabalho dentro do container, em que o mesmo acaba criando o diretório automaticamente se ele não existir.

```
WORKDIR /app
```

Código-fonte 7 – Exemplo de comando que define o diretório de trabalho no container via Dockerfile

Fonte: Elaborado pelo autor (2024)

- **RUN:** serve para executar comandos, podendo ser um simples 'echo' até comandos de instalação de dependências. É o caso a seguir, para executar o Streamlit dentro de um container com a imagem base de exemplo. É necessário atualizar o apt-get e instalar o gcc.

```
RUN apt-get update && \  
    apt-get install -y gcc
```

Código-fonte 8 – Exemplo de comando que baixa dependências necessárias para a construção de imagem via Dockerfile

Fonte: Elaborado pelo autor (2024)

- **COPY:** serve para copiar arquivos do diretório host (ou máquina local) para o container. No exemplo a seguir, estamos copiando os arquivos de um determinado diretório do host para a pasta /app que é o nosso diretório de trabalho (WORKDIR) dentro do container.

```
COPY . /app
```

Código-fonte 9 – Exemplo de comando que copia dados do diretório local para outro diretório dentro do container

Fonte: Elaborado pelo autor (2024)

- **RUN:** como dito anteriormente, serve para executar comandos durante o processo de construção da imagem. No exemplo a seguir, estamos instalando as dependências para a execução do Streamlit dentro do container.

```
RUN pip install -r requirements.txt
```

Código-fonte 10 – Exemplo de comando que baixa dependências da aplicação python via Dockerfile durante a criação da imagem

Fonte: Elaborado pelo autor (2024)

- **HEALTHCHECK:** esse comando serve para verificar a saúde do container. Caso algum erro venha a ocorrer, esse comando será acionado e o container será desativado. No comando a seguir, estamos verificando o localhost na porta 8080 definida pelo comando EXPOSE.
 - Além disso repare que estamos, na mesma linha, executando dois comandos diferentes, sendo eles HEALTHCHECK e CMD.

```
HEALTHCHECK CMD curl --fail http://localhost:8080/ || exit 1
```

Código-fonte 11 – Exemplo de comando que verifica a saúde do container após o processo de criação do container

Fonte: Elaborado pelo autor (2024)

- **VOLUME:** serve para criar um ponto de montagem somente dentro do container. É nossa responsabilidade, no momento de construção da imagem, sendo via Dockerfile ou Docker Compose, realizar a vinculação com esse volume.

```
VOLUME ["/data"]
```

Código-fonte 12 – Exemplo de comando que cria um volume dentro do container via Dockerfile

Fonte: Elaborado pelo autor (2024)

- **EXPOSE:** serve para definirmos qual porta queremos utilizar para comunicação externa. Entretanto, ainda assim, é necessário passar o parâmetro -p durante os comandos docker (docker run ou docker create) ou

no arquivo do Docker Compose para realizar a vinculação da porta do host com a porta do container.

```
EXPOSE 8080
```

Código-fonte 13 – Exemplo de comando que expõe a porta que queremos utilizar para comunicação externa via Dockerfile

Fonte: Elaborado pelo autor (2024)

- **ENTRYPOINT**: serve para definir um comando padrão e “estático”, que sempre será executado quando o container é inicializado.
 - Um outro exemplo que poderíamos utilizar seria **ENTRYPOINT** **[“streamlit”, “run”]** somente e inserir **“app.py”** no comando **CMD**.

```
ENTRYPOINT ["streamlit", "run", "app.py"]
```

Código-fonte 14 – Exemplo de comando que executa comandos padrões que sempre serão executados quando um container é inicializado via Dockerfile

Fonte: Elaborado pelo autor (2024)

- **CMD**: serve para passarmos parâmetros adicionais durante a execução do container, por exemplo. Se não definirmos esse comando no arquivo Dockerfile, implicitamente, por padrão, ele fica da seguinte maneira.

```
CMD []
```

Código-fonte 15 – Exemplo de comando que executa comandos adicionais para execução de containers via Dockerfile

Fonte: Elaborado pelo autor (2024)

Docker Compose

O Docker Compose às vezes é confundido com o Docker, porém ele é uma ferramenta de gerenciamento de containers para a construção de ambientes mais robustos. Já o Docker em si é a ferramenta principal em que realizamos download de imagens, criação e remoção de containers, criação de volumes etc.

Para utilizarmos essa ferramenta, devemos primeiro criar um arquivo chamado **docker-compose.yml** ou **docker-compose.yaml** e adicionar alguns comandos que vão servir como guia para criação e orquestração de serviços. Com poucos comandos

podemos construir N imagens e subir uma arquitetura de sistemas com um único comando.

O Docker Compose é um facilitador para replicação de sistemas, em que o mesmo ambiente que tenho em minha máquina local será executado sem complicações em outros ambientes. A seguir temos um exemplo de arquivo **docker-compose.yml**:

```
version: '3.8'

services:
  mongo:
    image: mongo
    ports:
      - "27017:27017"
    volumes:
      - ./data/mongodb:/data/db
    environment:
      MONGO_INITDB_ROOT_USERNAME: root
      MONGO_INITDB_ROOT_PASSWORD: root
    deploy:
      resources:
        limits:
          memory: 1g
    labels:
      kompose.service.type: "LoadBalancer"
    networks:
      - ml-network

  streamlit:
    image: python:3.8-slim
    container_name: ml_streamlit
    build:
      context: ./app
      dockerfile: Dockerfile
    ports:
      - "8080:8080"
    volumes:
      - ./app:/app
    deploy:
      resources:
        limits:
          memory: 500m
    labels:
      kompose.service.type: "LoadBalancer"
    depends_on:
      - mongo
    networks:
      - ml-network

networks:
  ml-network:
```

Código-fonte 16 – Exemplo de arquivo docker-compose.yml
Fonte: Elaborado pelo autor (2024)

Esse é o arquivo declarativo (docker-compose.yml) de comandos para gerenciamento de containers e imagens. Agora iremos abordar o que cada linha faz para uma melhor compreensão do arquivo.

- **version:** serve para especificar a versão do Docker Compose a ser utilizada.

```
version: '3.8'
```

Código-fonte 17 – Exemplo de version
Fonte: Elaborado pelo autor (2024)

- **services:** é uma chave principal na estrutura do arquivo docker-compose.yml que serve para definir N serviços (ou containers) que queremos utilizar em nosso projeto. Sendo assim, temos dois serviços (containers), sendo eles 'mongo' e 'streamlit'.

```
version: '3.8'

services:
  mongo:
    ...

  streamlit:
    ...
```

Código-fonte 18 – Exemplo de services
Fonte: Elaborado pelo autor (2024)

- **mongo:** é o nome do serviço (container) que definimos para o banco, é algo flexível no qual você tem a liberdade de escolher qual nome deseja dar ao serviço.

```
version: '3.8'

services:
  mongo:
    ...
```

Código-fonte 19 – Exemplo de mongo
Fonte: Elaborado pelo autor (2024)

- **image:** é onde selecionamos a imagem que queremos utilizar em nosso serviço (ou container).

```
mongo:
  image: mongo
```

Código-fonte 20 – Exemplo de image
Fonte: Elaborado pelo autor (2024)

- **ports:** é onde vinculamos qual porta queremos utilizar para comunicação externa. Com o Docker Compose, não precisamos de um comando adicional para realizar a vinculação de portas, pois isto já está declarado no arquivo.

```
ports:
  - "27017:27017"
```

Código-fonte 21 – Exemplo de ports
Fonte: Elaborado pelo autor (2024)

- **volumes:** essa chave faz a vinculação do diretório local com o diretório do container. Também não é necessário passar o parâmetro -v para vinculação, pois ele já está declarado no arquivo.

```
volumes:
  - ./data/mongodb:/data/db
```

Código-fonte 22 – Exemplo de volumes
Fonte: Elaborado pelo autor (2024)

- **environment:** podemos definir variáveis de ambiente no Dockerfile e no Docker Compose também, mas apenas se houver Dockerfile daquele determinado container. A seguir vamos definir as variáveis de usuário e senha do mongodb.

```
environment:
  MONGO_INITDB_ROOT_USERNAME: root
  MONGO_INITDB_ROOT_PASSWORD: root
```

Código-fonte 23 – Exemplo de environment
Fonte: Elaborado pelo autor (2024)

- **deploy:** nessa chave, definimos configurações para deploy no Docker Swarm. Aqui estamos definindo que o container tem um limite de recursos, sendo ele 1GB (Gigabyte) de memória.

```
deploy:
  resources:
    limits:
      memory: 1g
```

Código-fonte 24 – Exemplo de deploy
Fonte: Elaborado pelo autor (2024)

- **labels:** assim como no Dockerfile, no Docker Compose a chave 'labels' serve para adicionar metadados ao serviço. A seguir, estamos definindo o label para uma ferramenta chamada Kompose que tem como função converter o arquivo docker-compose.yml para criação de ambientes de homologação ou produção no kubernetes.

```
labels:
  kompose.service.type: "LoadBalancer"
```

Código-fonte 25 – Exemplo de labels
Fonte: Elaborado pelo autor (2024)

- **network:** serve para definir as redes às quais o serviço pertence (vamos abordar melhor esse assunto posteriormente). A seguir estamos definindo que o container está dentro da rede ml-network.

```
networks:
  ml-network:
```

Código-fonte 26 – Exemplo de network
Fonte: Elaborado pelo autor (2024)

Agora, repare no segundo serviço que se chama **streamlit**. Nele temos algumas diferenças, pois é uma aplicação que utiliza um Dockerfile para a construção do container.

```
streamlit:
  image: python:3.8-slim
  container_name: ml_streamlit
  build:
    context: ./app
    dockerfile: Dockerfile
  ports:
    - "8080:8080"
  volumes:
    - ./app:/app
  deploy:
    resources:
      limits:
        memory: 500m
  labels:
    kompose.service.type: "LoadBalancer"
  depends_on:
    - mongo
  networks:
    - ml-network
```

Código-fonte 27 – Exemplo de streamlit
Fonte: Elaborado pelo autor (2024)

Para não ficar repetitivo, vamos falar das chaves que ainda não conhecemos e que estão nesse serviço.

- **container_name:** serve para adicionar um nome ao container, parecido com o parâmetro **--name**. Podemos utilizar o namespace para realizar algumas operações manuais se necessário.

```
container_name: ml_streamlit
```

Código-fonte 28 – Exemplo de container_name
Fonte: Elaborado pelo autor (2024)

- **build:** serve para indicar o diretório em que o arquivo Dockerfile se encontra. Então repare que 'context' serve para selecionar o diretório e 'dockerfile' para indicar o nome do arquivo que se encontram as instruções do Dockerfile.

```
build:
  context: ./app
  dockerfile: Dockerfile
```

Código-fonte 29 – Exemplo de build
Fonte: Elaborado pelo autor (2024)

- **networks:** por último, mas não menos importante, temos a definição da rede (que vamos abordar melhor posteriormente) que engloba toda a nossa arquitetura de containers.

```
version: '3.8'

services:
  ...

networks:
  ml-network:
```

Código-fonte 30 – Exemplo de networks
Fonte: Elaborado pelo autor (2024)

Docker Compose: extensão de arquivos

No Docker Compose é possível estender arquivos para quando temos vários serviços dentro de um ecossistema, o que possibilita reutilizar configurações, criando arquivos principais nos quais sub-arquivos são chamados. A seguir, um exemplo disso.

```
services:
  webapp:
    extends:
      file: common.yaml
      service: app
    command: /code/run_web_app
    ports:
      - 8080:8080
    depends_on:
      - queue
      - db

  queue_worker:
    extends:
      file: common.yaml
      service: app
    command: /code/run_worker
    env_file:
      - ../commons/container.env
    depends_on:
      - queue
```

Código-fonte 31 – Exemplo de um arquivo docker-compose.yml que utiliza a técnica de extensão de configurações (1)
Fonte: Docker Docs (2024)

O arquivo anterior está chamando o sub-arquivo **common.yaml** e selecionando o serviço **app** para estender as configurações no arquivo **docker-compose.yml** no serviço **webapp**. A seguir temos a configuração do sub-arquivo:

```
services:
  app:
    build: .
    environment:
      CONFIG_FILE_PATH: /code/config
      API_KEY: my-key
    cpu_shares: 5
```

Código-fonte 32 – Exemplo de um arquivo docker-compose.yml que a utiliza técnica de extensão de configurações (2)

Fonte: Docker Docs (2024)

Podemos ver que no sub-arquivo temos algumas configurações inexistentes no arquivo docker-compose.yml, que podem ser variáveis de ambientes, portas, pontos de montagem, pontos de criação de imagens (Dockerfiles diferentes) etc.

Um cenário em que essa abordagem se encaixa muito bem é quando se torna necessário criar ambientes de desenvolvimento, homologação e produção.

Para isso, é possível também criarmos três arquivos diferentes (utilizando o exemplo dos três ambientes anteriores), podendo eles ser: docker-compose-local.yml, que estende common.yaml para o ambiente local; docker-compose-qa.yml, que estende algumas configurações do common.yaml para o ambiente de homologação; e docker-compose-prd.yml, que estende algumas configurações do common.yaml para o ambiente de produção.

Podemos chamar esses arquivos com o parâmetro -f, conforme exemplo a seguir.

```
$ docker compose <build|up|down> -f docker-compose-<env>.yaml
```

Código-fonte 33 – Exemplo de comando docker compose para chamar arquivos principais e sub-arquivos de extensão com o parâmetro -f e demonstração de que podemos utilizar os sub-comandos build e up na chamada desses arquivos

Fonte: Elaborado pelo autor (2024)

Docker Compose: mesclagem de arquivos

No Docker Compose podemos utilizar a técnica de extensão de arquivos explicitando diretamente no arquivo de configuração **docker-compose.yml**, mas

também podemos utilizar a mesclagem de arquivos empregando **parâmetros -f**. Para exemplificar, temos um **docker-compose-qa.yml** a seguir.

```
webapp:
  image: examples/web
  ports:
    - "8000:8000"
  volumes:
    - "/data"
```

Código-fonte 34 – Exemplo de um arquivo docker-compose.yml que utiliza técnica de mesclagem
Fonte: Docker Docs (2024)

Suponhamos que temos o arquivo de configuração X que tem algumas configurações que queremos adicionar na chamada do nosso arquivo principal de ambiente de homologação.

```
webapp:
  environment:
    - DEBUG=1
```

Código-fonte 35 – Exemplo de um arquivo yaml de configuração usado na técnica de mesclagem
Fonte: Docker Docs (2024)

Para realizar a técnica de mesclagem e subir nosso ambiente de homologação, como dito anteriormente, podemos utilizar o **parâmetro -f** da seguinte forma:

```
$ docker compose -f config.yml -f docker-compose.yml <up|build>
```

Código-fonte 36 – Exemplo de comando docker compose usando técnica de mesclagem
Fonte: Docker Docs (2024)

Sendo assim, como resultado temos:

```
webapp:
  image: examples/web
  ports:
    - "8000:8000"
  volumes:
    - "/data"
  environment:
    - DEBUG=1
```

Código-fonte 37 – Exemplo de um arquivo docker-compose.yml que utiliza técnica de mesclagem
Fonte: Docker Docs (2024)

E assim podemos utilizar a técnica de mesclagem de arquivos no Docker Compose, reutilizando arquivos em ambientes grandes de forma simplificada e organizada.

Arquivo .dockerignore

Esse é um arquivo que serve muito bem para a otimização de imagens ou ambientes, pois serve para ignorar arquivos ou diretórios indesejados ou desnecessários.

```
file
path/file
path/subpath/file
```

Código-fonte 38 – Exemplo de arquivo .dockerignore
Fonte: Elaborado pelo autor (2024)

Kompose

Na aula utilizamos o Kompose para converter o arquivo docker-compose.yml para o formato Kubernetes de forma automática. Essa ferramenta permite facilitar a transição de ambientes de desenvolvimento para ambientes de homologação e produção.

Como estamos utilizando Docker, em vez de instalar o **Kompose** em nossa máquina, a ideia foi utilizar o repositório do GitHub da ferramenta e transformar em uma imagem para executar os comandos dentro do nosso container Kompose e do diretório em que se encontra o arquivo **docker-compose.yml**. Temos um exemplo a seguir:

```
$ docker build -t kompose https://github.com/kubernetes/kompose.git\#main && \
docker run --rm -it -v ./opt kompose sh -c "cd /opt && kompose convert -f docker-
compose-prd.yml"
```

Código-fonte 39 – Exemplo de comando para construir uma imagem a partir de um repositório
Fonte: Elaborado pelo autor (2024)

No comando anterior, estamos criando a imagem a partir da branch main do repositório da ferramenta Kompose e logo em seguida convertendo a imagem em container e executando com o parâmetro `-rm`. Ele serve para excluir o container após encerrar a execução dos comandos que passamos no parâmetro `-c` para a conversão do arquivo docker-compose.yml para o formato Kubernetes.

Após a execução do container, arquivos de service.yaml e deployment.yaml serão gerados para cada serviço dentro do arquivo docker-compose.yaml com suas respectivas configurações e portas.

Por fim, podemos utilizar o comando de deploy para os dois arquivos de cada serviço do docker-compose.yml, ficando da seguinte forma:

```
$ kubectl apply -f ./<nome_serviço>-deployment.yaml  
$ kubectl apply -f ./<nome_serviço>-service.yaml
```

Código-fonte 40 – Exemplo de comandos para a realização de deploy
Fonte: Elaborado pelo autor (2024)

Google Kubernetes Engine (GKE)

No mundo dos containers temos o Kubernetes, que é um ambiente no qual podemos operar aplicações de forma escalável e containerizada.

E na infraestrutura da Google temos o GKE (Google Kubernetes Engine), que visa executar aplicações containerizadas, oferecendo uma plataforma que permite configurar infraestrutura, hardware, segurança, rede etc (não iremos abordar a fundo esse tópico no módulo de containers, mas é importante saber como aplicações containerizadas funcionam em um ambiente de homologação ou produção).

No Hands on, utilizamos um projeto simples de recomendação de filmes usando streamlit e outras ferramentas. A ideia principal foi realizar o processo de deploy em um ambiente para aplicações containerizadas, em que tivemos sucesso realizando alguns passos como:

- Criação das imagens do streamlit e do jupyter notebook.
- Criação das tags das imagens do streamlit e do jupyter notebook passando o nosso **project id** que pegamos do Google Cloud Platform.
- Realizamos o deploy das nossas imagens no Artifact Registry para versionar e utilizar de forma consistente e segura no ambiente de produção.
- Em seguida realizamos a conversão do docker-compose-prd.yml para os arquivos necessários que o Kubernetes espera usando o Kompose.

- Por fim, executamos o comando de deploy para cada arquivo de configuração do Kubernetes.

Após essa sequência de passos até o deploy, executamos o comando a seguir para visualizar nossos serviços em produção:

```
$ kubectl get services
```

Código-fonte 41 – Exemplo de comando que exibe todos os serviços que estão no nosso ambiente kubernetes

Fonte: Elaborado pelo autor (2024)

Como resultado tivemos para cada serviço um serviço para a aplicação containerizada e outro para comunicação, sendo que com <serviço>-tcp podemos acessar via external-ip e port da seguinte forma <external-ip>:<port>:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
jupyter	ClusterIP	34.118.239.22	<none>	8888/TCP	4d9h
jupyter-tcp	LoadBalancer	34.118.230.62	104.196.56.139	8888:30604/TCP	4d9h
kubernetes	ClusterIP	34.118.224.1	<none>	443/TCP	4d10h
mongo	ClusterIP	34.118.236.124	<none>	27017/TCP	4d9h
mongo-tcp	LoadBalancer	34.118.232.14	34.75.74.178	27017:32544/TCP	4d9h
mongodb-compass	ClusterIP	34.118.231.177	<none>	8081/TCP	4d9h
mongodb-compass-tcp	LoadBalancer	34.118.239.125	34.75.48.199	8081:30159/TCP	4d9h
streamlit	LoadBalancer	34.118.226.215	34.73.106.91	8501:30330/TCP	4d9h
streamlit-tcp	LoadBalancer	34.118.237.58	35.231.19.44	8080:32506/TCP	4d9h

Figura 1 - Demonstração do resultado do comando que exibe todos os serviços do nosso ambiente kubernetes

Fonte: Elaborado pelo Autor (2024)

Na sequência utilizamos o comando a seguir para poder visualizar a saúde das aplicações no GKE (**pods** é um grupo de um ou mais containers que trabalham em conjunto):

```
$ kubectl get pods
```

Código-fonte 42 – Exemplo de kubectl get pods

Fonte: Elaborado pelo autor (2024)

Executando o código anterior tivemos como resultado algumas informações dos pods, como por exemplo o status do container que nos indica se está parado, com erro ou em execução, entre outras informações.

NAME	READY	STATUS	RESTARTS	AGE
jupyter-c5f94f474-6lx5x	1/1	Running	1 (3h4m ago)	3d21h
mongo-94f569b7d-f6l47	1/1	Running	0	3d21h
mongodb-compass-c96f46d5f-zblpv	1/1	Running	0	3d21h
streamlit-8b99d8c4b-p86m9	1/1	Running	1 (3d21h ago)	3d21h

Figura 2 - Demonstração do resultado do comando que exibe todos os pods do nosso ambiente gke
Fonte: Elaborado pelo Autor (2024)

Em seguida, executamos o comando para visualização de logs dos containers para se certificar de que estava tudo bem com nossas aplicações:

```
$ kubectl logs streamlit-8b99d8c4b-p86m9
```

Código-fonte 43 – Exemplo de comando para visualizar os logs do pod
Fonte: Elaborado pelo autor (2024)

Executando o código anterior tivemos como resultado a mensagem padrão do streamlit quando executamos em ambiente local. Entretanto, temos nosso **external-ip** do GKE que serve para substituir os endereços da figura a seguir como um IP de produção (sendo bem parecido com o que acontece quando realizamos um deploy no streamlit):

```
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8080
Network URL: http://10.96.1.27:8080
External URL: http://35.237.72.210:8080
```

Figura 3 - Demonstração do resultado do comando que exibe log dos pods
Fonte: Elaborado pelo Autor (2024)

E por fim, executamos outro comando que tem como função exibir informações úteis sobre o pod escolhido. Com o comando a seguir, podemos visualizar detalhes como recursos, local de download de imagem etc:

```
$ kubectl describe pod streamlit-8b99d8c4b-p86m9
```

Código-fonte 44 – Exemplo de comando para visualizar detalhes do pod
Fonte: Elaborado pelo autor (2024)

O QUE VOCÊ VIU NESTA AULA?

Nessa aula você aprendeu o que é Dockerfile e como utilizá-lo, além de otimizar imagens. Analisamos as boas práticas de criação de imagens e exploramos funcionalidades avançadas, além de realizar deploy de um modelo de análise de sentimentos usando Dockerfile e Streamlit no Cloud Run do Google Cloud Platform.

No que diz respeito ao Docker Compose aprendemos que esta é uma ferramenta de gerenciamento de containers para ambientes com alta escalabilidade. Além disso, exploramos funcionalidades avançadas utilizando volumes, variáveis de ambientes e outras ferramentas, como o Kompose, que tem como função converter arquivos docker-compose.yml para o formato Kubernetes.

Por fim, realizamos deploy de um ambiente criado via Docker Compose no Google Kubernetes Engine, que é um ambiente para aplicações containerizadas.

O último projeto de portfólio desse tópico teve como objetivo demonstrar o mundo real no mercado de trabalho, além de como podemos criar e realizar deploy em ambientes de homologação e/ou produção de aplicações que utilizam machine learning ou deep learning, utilizando em conjunto um banco de dados MongoDB para consulta e um modelo para predição de recomendação de filmes utilizando o SVD (algoritmo de deep learning).

Lembre-se que estamos disponíveis na nossa comunidade do Discord para tirar dúvidas e te ajudar. Além disso o conteúdo estará sempre disponível quando você sentir a necessidade de revisitá-lo!

REFERÊNCIAS

DOCKER DOCS. **Extend your Compose file.** 2024. Disponível em: <<https://docs.docker.com/compose/multiple-compose-files/extends/>>. Acesso em: 27 ago. 2024.

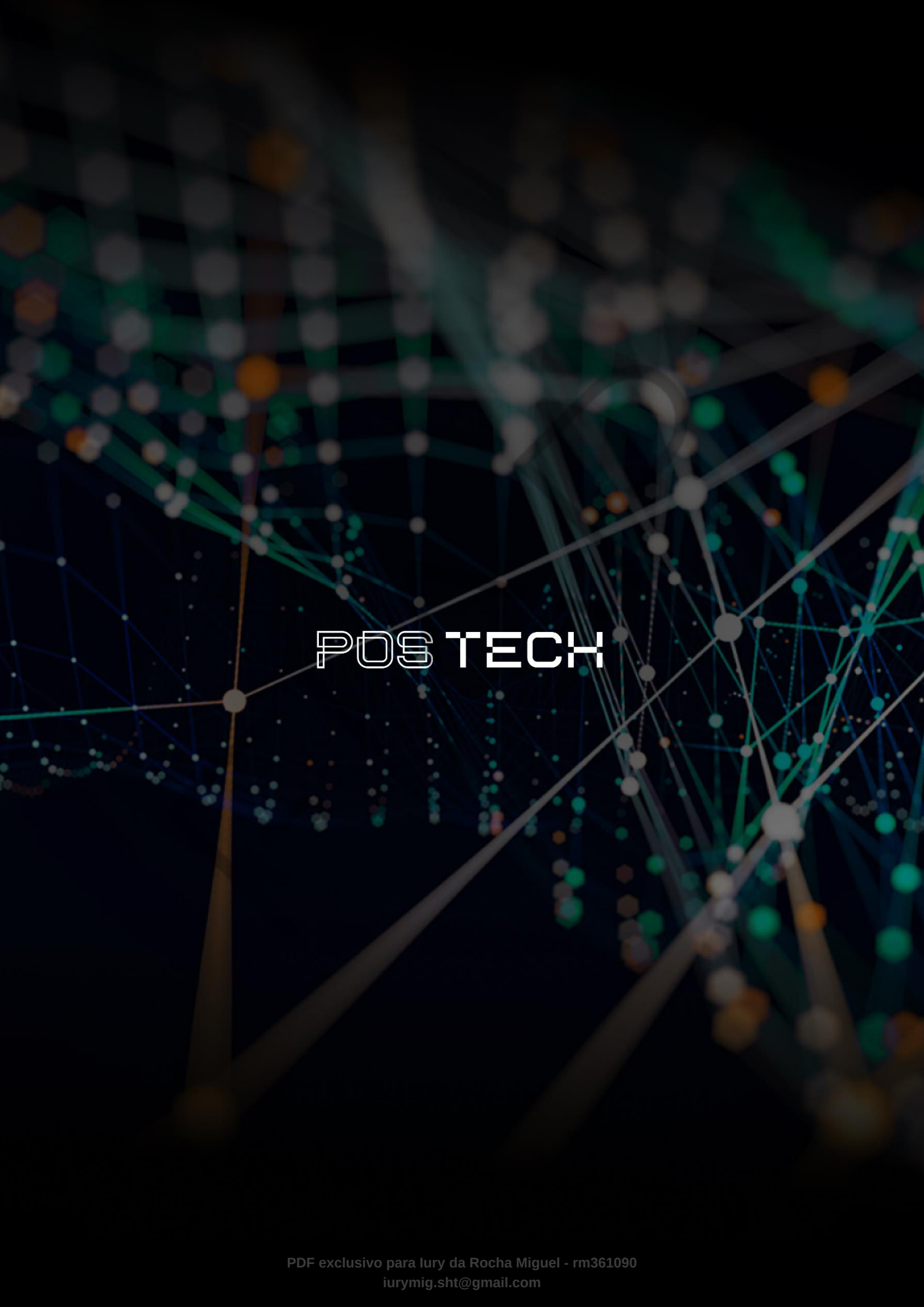
DOCKER DOCS. **Merge Compose files.** 2024. Disponível em: <<https://docs.docker.com/compose/multiple-compose-files/merge/>>. Acesso em: 27 ago. 2024.

GOOGLE CLOUD DOCS. **Visão geral do GKE.** 2024. Disponível em: <<https://cloud.google.com/kubernetes-engine/docs/concepts/kubernetes-engine-overview>>. Acesso em: 27 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: Kubernetes. Docker. Containers. Docker Compose. GKE. Google Kubernetes Engine. Cloud Run. GCP. Images Docker. Docker Hub.

EMSE



POSTECH